Teorema de Incompletitud de Gödel

Luis Vicente Cofré Osorio
Facultad de Matemáticas
Pontificia Universidad Católica de Chile

23 de marzo de 2026

Índice

1. Introducción **1**

2. Conceptos previos **2**

3. Teorías e Incompletitud de Gödel **5**

4. Maquinas de Turing y conceptos relacionados **9**

5. Demostración de la indecidbilidad de $Th\mathfrak{N}$ **20**

1. Introducción

En esta sesión del seminario de lógica matemática vamos a demostrar el teorema de incompletitud de Gödel usando la noción de las maquinas de turing. Primero abordaremos un estudio de teorías que finalizara con la demostración del teorema de incompletitud de Gödel como tal pero asumiremos que la teoría de los números naturales es indecidible, luego vamos a pasar por conceptos de maquinas de turing y computación y finalizaremos

con una demostración de que la teoría de los números naturales es indecidible usando maquinas de turing para que no haya nada sin demostrar.

2. Conceptos previos

Definición 2.1. Un vocabulario $\mathcal{L}$ consiste en tres conjuntos de simbolos

1. constantes $\{c_1, \cdots, c_l. \cdots\}$
2. funciones $\{f_1, \cdots, f_n, \cdots\}$
3. relaciones $\{R_1, \cdots, R_m, \cdots\}$

Donde a cada función y relación se le asigna un numero natural llamado aridad el cual es el numero de variables que acepta como argumento.

Ejemplo 1. *Un vocabulario para los naturales puede ser*$\mathcal{L} = \{0, 1, S, +, \cdot, <\}$ *donde* $0, 1$ *son constantes,* $S, +\cdot$ *son funciones y* $<$ *es una relación.*

Definición 2.2. Dado un vocabulario $\mathcal{L}$ definimos una $\mathcal{L}-$estructura $\mathfrak{A}$ (o estructura en logica de primer orden) como una tupla que contiene:

1. un dominio $A \neq \varnothing$.
2. para todo simbolo constante $c \in \mathcal{L}$ un elemento $c^{\mathfrak{A}} \in A$.
3. para cada función de aridad n $f \in \mathcal{L}$ una funcion $f^{\mathfrak{A}} : A^n \to A$.
4. para cada relación de aridad n $R \in \mathcal{L}$ una relación $R^{\mathfrak{A}} : A^n \to A$

A los elementos $c^{\mathfrak{A}}, f^{\mathfrak{A}}, R^{\mathfrak{A}}$ se les suele llamar interpretaciones de los símbolos respectivos, a la estructura se le denotara como $\mathfrak{A} = (A, c^{\mathfrak{A}}, \cdots, f^{\mathfrak{A}}, \cdots, R^{\mathfrak{A}}, \cdots)$.

Observación 1. Una forma fácil de entender esto es que se distingue entre el símbolo físico que usamos 1 y como varia lo que significa según que estemos estudiando, por ejemplo en los naturales es el numero 1 pero en un grupo se usa 1 para representar al elemento neutro, ahí tenemos que el mismo símbolo tiene en dos estructuras diferentes la misma forma de representarlo aunque a priori representen cosas distintas.

Ejemplo 2. *dado el vocabulario* $\mathcal{L} = \{0, 1, S, +, \cdot, <\}$ *se puede representar a los naturales como la* $\mathcal{L}-$*estructura*

$$\mathfrak{N} = (\mathbb{N}, 0^{\mathfrak{A}}, 1^{\mathfrak{A}}, S^{\mathfrak{A}}, +^{\mathfrak{A}}, \cdot^{\mathfrak{A}}, <^{\mathfrak{A}})$$

Definición 2.3. dada una $\mathcal{L}$-estructura $\mathfrak{A}$ de dominio A decimos que φ es consecuencia logica de $\mathfrak{A}$ si es verdad en dicha estructura y se define de manera inductiva de la siguiente manera:

1. Para toda tautologia $\mathfrak{A} \models \top$.
2. Para toda contradiccion $\mathfrak{A} \models \neg\bot$ (tambien denotado como $\mathfrak{A} \not\models \bot$).
3. $\mathfrak{A} \models \neg\varphi$ si $\mathfrak{A} \not\models \varphi$.
4. $\mathfrak{A} \models \varphi \vee \psi$ si $\mathfrak{A} \models \varphi$ o $\mathfrak{A} \models \psi$.
5. $\mathfrak{A} \models \varphi \wedge \psi$ si $\mathfrak{A} \models \varphi$ y $\mathfrak{A} \models \psi$.
6. $\mathfrak{A} \models \varphi \rightarrow \psi$ si $\mathfrak{A} \not\models \varphi$ o $\mathfrak{A} \models \psi$.
7. $\mathfrak{A} \models \varphi \leftrightarrow \psi$ si $\mathfrak{A} \models \varphi$ y $\mathfrak{A} \models \psi$ o $\mathfrak{A} \not\models \varphi$ y $\mathfrak{A} \not\models \psi$.
8. $\mathfrak{A} \models \forall x\varphi(x)$ si para todo $a \in A$ $\mathfrak{A} \models \varphi(a)$ (de forma analoga se define en caso de tener multiples variables).
9. $\mathfrak{A} \models \exists x\varphi(x)$ si existe $a \in A$ tal que $\mathfrak{A} \models \varphi(a)$ (de forma analoga se define en caso de tener multiples variables).

Como todas las formulas en lógica de primer orden se construyen de forma inductiva a partir de estos casos tenemos que esta noción esta definida para toda formula en lógica de primer orden.

Observación 2. Cuando decimos $\mathfrak{A} \models \varphi$ se omite un detalle por simplicidad, pero esto significa que si φ tiene n-variables $x_1, \cdots, x_n$ significa que existen $a_1, \cdots, a_n \in A$ tal que $\varphi(a_1, \cdots, a_n)$ es verdad en la estructura. Es decir se pide que exista al menos una asignación de valores que la haga verdadera.

El caso de las formulas atómicas se omite porque en estos casos las formulas atómicas serian las funciones, relaciones y constantes y estas se satisfacen en la estructura si existen

al menos unos valores que la hacen verdad en ella, por ejemplo la noción de $x < y$ es consecuencia lógica si encuentro dos tuplas de naturales que la hacen verdad, lo mismo con las funciones.

Definición 2.4. Dado un conjunto de oraciones Σ sobre un vocabulario $\mathcal{L}$ decimos que es satisfacible si existe $\mathfrak{A}$ $\mathcal{L}$- estructura tal que $\mathfrak{A} \models \Sigma$.

Definición 2.5. Dado un conjunto Σ de oraciones y una oración φ diremos que $\Sigma \models \varphi$ si para toda $\mathfrak{A}$ estructura tal que $\mathfrak{A} \models \Sigma$ se tiene que $\mathfrak{A} \models \varphi$

Definición 2.6. Dado un conjunto de oraciones Σ y φ una oración decimos que φ es demostrable a partir de Σ si existe una secuencia de formulas $\rho_1, \cdots, \rho_n$ tal que la primera es una o varias formulas de Σ, la ultima es φ y el paso de ρ_i a ρ_{i+1} se logra aplicando reglas de deducción e inferencia a ρ_i y lo denotamos como $\Sigma \vdash \varphi$.

Teorema 2.7 (Correctitud y Completitud)**.** *para todo conjunto de formulas Σ y para toda formula φ se tiene que*

$$\Sigma \vdash \varphi \Longleftrightarrow \Sigma \models \varphi$$

Observación 3. Este teorema es muy importante porque cuando uno estudia sistemas lógicos, le gustaría saber cuando un sistema deductivo es bueno, y hay dos criterios en los que a uno le gustaría fijarse.

Primero esta la completitud, la cual nos dice que si algo es demostrable en mi sistema, entonces es necesariamente verdadero en este, expresado de la siguiente manera:

$$\Sigma \vdash \varphi \implies \Sigma \models \varphi.$$

Luego tenemos la correctitud, la cual nos dice que si algo es verdadero en mi sistema formal, entonces puedo construirme una demostración de que es verdadero, expresado como sigue:

$$\Sigma \models \varphi \implies \Sigma \vdash \varphi.$$

Afortunadamente en lógica de primer orden estas nociones son equivalentes entre si.

3. Teorías e Incompletitud de Gödel

Definición 3.1. Sea Σ un conjunto de oraciones sobre un vocabulario $\mathcal{L}$, decimos que Σ es una teoría si:

- Σ es satisfacible
- Es cerrado bajo consecuencia lógica, es decir si φ es una oración tal que $\Sigma \models \varphi$, entonces $\varphi \in \Sigma$

Definición 3.2. Sea $\mathfrak{A}$ una $\mathcal{L}$-estructura, definimos la teoría de $\mathfrak{A}$ como:

$$Th\mathfrak{A} = \{\varphi \ \mathcal{L}\text{-oraciones} : \mathfrak{A} \models \varphi\}$$

se puede entender como las proposiciones que podemos deducir a partir de $\mathfrak{A}$

Ejemplo 3. *si consideramos $\mathfrak{N}$ la estructura usual de los números naturales entonces $Th\mathfrak{N}$ es la aritmética. (no es la teoría de números por unas razónes que veremos mas adelante).*

Definición 3.3. decimos que una teoría Σ es completa si para toda oración φ tenemos que $\varphi \in \Sigma$ o $\varphi \notin \Sigma$.

Proposición 3.4. *Para toda $\mathcal{L}-$estructura $\mathfrak{A}$ tenemos que $Th\mathfrak{A}$ es una teoría completa.*

Demostración. Vamos a demostrar por partes que es una teoría y luego que es completa.

- Para ver que es teoría veamos que es satisfacible y cerrada bajo consecuencia lógica.

  Primero notemos que como $\mathfrak{A} \models \varphi$ para todo $\varphi \in Th\mathfrak{A}$ tenemos que por definicion $\mathfrak{A} \models Th\mathfrak{A}$.

  Para ver que es cerrada bajo consecuencia lógica notemos que si $Th\mathfrak{A} \models \varphi$, como $\mathfrak{A} \models Th\mathfrak{A}$ por definicion tenemos que $\mathfrak{A} \models \varphi$ lo cual implica $\varphi \in Th\mathfrak{A}$ por definicion.

- Para ver que es completa, notemos que si φ es una oracion arbitraria tenemos que $\mathfrak{A} \models \varphi$ o $\mathfrak{A} \not\models \varphi$ lo cual implica $\mathfrak{A} \models \varphi$ o $\mathfrak{A} \models \neg\varphi$, así por definicion tenemos que $\varphi \in Th\mathfrak{A}$ o $\neg\varphi \in Th\mathfrak{A}$ teniendo que es completa.

■

Corolario 3.5. *$Th\mathfrak{N}$ es completa.*

Si bien esta noción de teoría suena razónable, no se asemeja a lo que la mayoría de los matemáticos entendemos. Siempre solemos entender la matemática como un conjunto de axiomas que, junto con reglas de deducción e inferencia, construye el cuerpo teórico que estudiamos, lo que nos da la intuición de definir las teorías generadas por un conjunto axiomático.

Definición 3.6. Dado un conjunto AX de $\mathcal{L}$-oraciones satisfacible definimos a la teoría generada por AX como el conjunto

$$ThAX = \{\varphi \ \mathcal{L}\text{-oraciones } : AX \models \varphi\}$$

Observación 4.

1. si AX son los 5 axiomas de Euclides tenemos que $ThAX$ es la geometría euclediana, si Gr son los axiomas de grupos tenemos que $ThGr$ es la teoría de grupos, si PA es la aritmética de Peano, tenemos que $ThPA$ es la teoría de números.

2. Uno puede ver que $ThAX$ es una teoría, para ello notar que como AX es satisfacible existe una estructura en lógica de primer orden $\mathfrak{A}$ tal que $\mathfrak{A} \models AX$ luego para todo $\varphi \in ThAX$ tenemos que $AX \models \varphi$ lo que por definicion junto con el hecho de que $\mathfrak{A} \models AX$ nos permite concluir que $\mathfrak{A} \models \varphi$, lo cual implica que $\mathfrak{A} \models ThAX$ i.e es satisfacible.

   Para ver que es cerrado bajo consecuencia lógica tomamos una oración φ tal que $ThAX \models \varphi$, para probar que $\varphi \in ThAX$ es suficiente demostrar que $AX \models \varphi$ o equivalentemente que si $\mathfrak{A} \models AX$ entonces $\mathfrak{A} \models \varphi$. Si $\mathfrak{A} \models AX$ razonando como en el punto anterior tenemos que $\mathfrak{A} \models ThAX$, así tenemos que por definición $ThAX \models \varphi$ implica que $\mathfrak{A} \models \varphi$ lo cual prueba lo deseado concluyendo que es una teoría.

3. la razón de porque existe definimos la teoría de una estructura lógica de primer orden y la teoría de un conjunto axiomático como cosas diferentes es porque lo son, poseen propiedades fundamentales diferentes las unas de las otras, una que es muy interesante es que las primeras siempre son teorías completas mientras que las teorías generadas por conjuntos axiomáticos no siempre son completas lo que las hace mas difíciles de manejar a nivel lógico.

Proposición 3.7. *La teoría de grupos no es completa.*

Demostración. para demostrarlo basta encontrar una proposición φ tal que $\varphi \notin ThGr$ y $\neg\varphi \notin ThGr$. Definiremos dicha proposición como $\varphi = \forall x \forall y (x \cdot y = y \cdot x)$.

Notar que $\varphi \notin ThGr$ si y solo si $Gr \nvDash \varphi$ lo cual ocurre si existe una estructura tal que $\mathfrak{A} \models Gr$ y $\mathfrak{A} \nvDash \varphi$, como $\mathfrak{A} \models Gr$ tenemos que la estructura debe ser un grupo, por lo tanto para demostrar que $\varphi \notin ThGr$ es suficiente con encontrar un grupo no abeliano, para ello es suficiente considerar $(GL_n(\mathbb{R}), \cdot)$ con $\cdot$ el producto de matrices, para demostrar que $\neg\varphi \notin ThGr$ basta con encontrar un grupo que si sea abeliano, para ello es suficiente considerar $(\mathbb{Z}, +)$. ■

Definición 3.8. decimos que una teoría Σ es decidible si existe un algoritmo que permite determinar para toda oración φ si $\varphi \in \Sigma$ o $\varphi \notin \Sigma$ (se define de modo análogo que un conjunto de oraciones sea decidible).

Observación 5. De momento se definirá la decidibilidad de esa manera sin profundizar mucho en lo que quiere decir algoritmo, todo eso sera profundizado a su debido tiempo en la sección de maquinas turing.

Definición 3.9. Dada una teoría Σ sobre $\mathcal{L}$, decimos que es recursivamente axiomatizable si existe un conjunto AX de $\mathcal{L}-$oraciones satisfacible y decidible tal que

$$\Sigma = ThAX$$

Teorema 3.10. *Si Σ es una teoría completa y recursivamente axiomatizable entonces Σ es decidible*

Demostración. sea AX un conjunto axiomático que es satisfacible y decidible de $\mathcal{L}$-oraciones tal que $\Sigma = ThAX$, vamos a construir un algoritmo que permita determinar para toda oracion φ si $\varphi \in \Sigma$ o $\varphi \notin \Sigma$.

Recordemos que $\Sigma = ThAX = \{\varphi : AX \models \varphi\} = \{\varphi : AX \vdash \varphi\}$ (aplicando el teorema 2.7, aprovecharemos este teorema para intercambiar ambas nociones cuando nos sea conveniente).

Tomamos la proposición φ y hacemos lo siguiente:

1. Como AX es decidible existe un algoritmo que nos permite determinar si $AX \vdash \varphi$ o $AX \nvdash \varphi$.
2. Si $AX \vdash \varphi$ entonces $\varphi \in ThAX = \Sigma$.
3. Si $AX \nvdash \varphi$, como Σ es completa tenemos que $\varphi \in \Sigma$ o $\neg\varphi \in \Sigma$ de donde tenemos dos casos:
   - Si $\neg\varphi \in \Sigma$, entonces tenemos que $\varphi \notin \Sigma$, para ver esto supongamos que $\varphi \in \Sigma = ThAX$ entonces $AX \vdash \varphi$, esto junto con el hecho de que $AX \nvdash \varphi$ nos dice que existe y no existe una demostracion desde AX hasta φ por lo que necesariamente $\varphi \notin \Sigma$
   - Si $\varphi \in \Sigma$ misma contradicción de arriba, por lo que no es posible

por lo que al final del proceso tenemos que $\varphi \in \Sigma$ o $\varphi \notin \Sigma$ concluyendo que es una teoría decidible. ■

Teorema 3.11. *si Σ es una teoría completa y Γ un conjunto de oraciones satisfacible tal que $\Sigma \subseteq \Gamma$ entonces tenemos que $\Sigma = \Gamma$.*

Demostración. Notar que basta demostrar que $\Gamma \subseteq \Sigma$, Si $\varphi \in \Gamma$ como Σ es completa tenemos que $\varphi \in \Sigma$ o $\neg\varphi \in \Sigma$, si es el primer caso estamos listos por lo que basta estudiar el segundo caso.

Si $\neg\varphi \in \Sigma$, como $\Sigma \subseteq \Gamma$ entonces tenemos que $\neg\varphi \in \Gamma$, como Γ es satisfacible existe estructura en lógica de primer orden $\mathfrak{A}$ tal que $\mathfrak{A} \models \Gamma$ por lo que para toda formula $f \in \Gamma$ tenemos que $\mathfrak{A} \models f$, en particular tenemos que $\mathfrak{A} \models \varphi$ y $\mathfrak{A} \models \neg\varphi$, concluyendo que $\mathfrak{A} \models \varphi \wedge \neg\varphi$ lo cual es una contradicción, por lo que necesariamente $\varphi \in \Sigma$ concluyendo que $\Gamma \subseteq \Sigma$. ■

Teorema 3.12. *$Th\mathfrak{N}$ es indecidible*

la demostración de este teorema no se hará en esta sección, una formulación moderna de la demostración utiliza maquinas de turing así que para facilitar la comprension se dejara para mas adelante y de momento se asumira como cierto.

Teorema 3.13 (Incompletitud de Gödel)**.** *Sea $\mathcal{L} = \{0, 1, S, +, \cdot, <\}$ y AX un conjunto decidible y satisfacible de $\mathcal{L}$-oraciones, entonces existe al menos una $\mathcal{L}$-oracion φ tal que*

$$\varphi \in Th\mathfrak{N} \text{ y } \varphi \notin ThAX$$

Observación 6.

- la conclusion del teorema equivale a que $Th\mathfrak{N} \nsubseteq ThAX$
- este teorema quiere decir que existen proposiciones en la aritmética tales que no se pueden deducir a partir de AX, por lo que para todo sistema axiomático que sea decidible y satisfacible existen proposiciones en la aritmética que no pueden ser demostradas a partir de ellos.
- como se vera mas adelante en la demostración, implica que $Th\mathfrak{N}$ no es recursivamente axiomatizable, por lo que de existir algún conjunto que axiomatice a la aritmética es inconsistente o bien es indecidible.
- Cualquier teoría matemática que sea lo suficientemente potente para tener aritmética heredara el teorema de incompletitud.
- Este teorema nos dice que existen proposiciones que pueden ser verdad pero que no existe ninguna demostración de que lo sean.

Demostración. Supongamos por contradicción que existe un conjunto axiomático decidible y satisfacible AX tal que $Th\mathfrak{N} \subseteq ThAX$ tenemos que como $ThAX$ es teoría, es satisfacible y por el corolario 3.5 tenemos que $Th\mathfrak{N}$ es completo, por lo que por el teorema 3.11 tenemos que $Th\mathfrak{N} = ThAX$ por lo que la aritmética puede ser axiomatizada por un conjunto decidible y satisfacible, por definición esto quiere decir que $Th\mathfrak{N}$ es recursivamente axiomatizable y completa, por el teorema 3.10 tenemos que es $Th\mathfrak{N}$ es decidible, obteniendo contradicción. ■

4. Maquinas de Turing y conceptos relacionados

En esta sección veremos la formulación de las maquinas de turing, su rol en la decidibilidad y los algoritmos y algunas propiedades sobre reducción, y también una demostración de la indecidibilidad del problema de la detención. Antes de comenzar con el tema destacare que existen múltiples formulaciones de la maquina de turing, la cinta en una sola dirección, multicinta, aquella que no permite volverse hacia atrás sino hacer un reset y las maquinas de turing cuánticas entre otros pero uno puede demostrar que todos esos modelos son equivalentes entre si, no nos centraremos en demostrar este hecho porque no es el objetivo de este trabajo, solo se asume como cierto y elegiremos una formulación de las maquinas de turing con la que trabajar.

Definición 4.1 (Nociones Básicas)**.** Comenzaremos por definir el set inicial en el que se mueve esta sección

1. Sea Σ un conjunto no vació, lo llamaremos alfabeto, por ejemplo $\Sigma = \{0, 1\}$ o $\Sigma = \{a, b, c\}$
2. Una palabra en el alfabeto Σ es una secuencia de elementos concatenados $w = w_1 w_2 \cdots w_n$ tal que $w_i \in \Sigma$, en los ejemplos anteriores tenemos que 001110 y $abbabc$ son palabras.
3. definimos Σ^* como el conjunto de todas las palabras que uno puede formar a partir de Σ, diremos que un lenguaje es cualquier $L \subseteq \Sigma$, en este caso $\mathbb{N} = \{0, 1\}^*$ es un lenguaje, el código binario y si Σ son todas las letras del abecedario un lenguaje posible es el español.

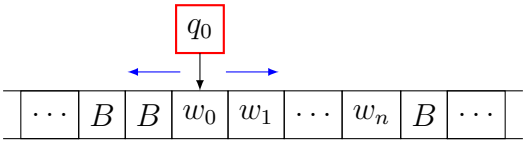
Definición 4.2. Una maquina de turing (o MT) determinista M es una tupla $M = (Q, A, \Gamma, q_0, F, \delta)$ donde:

1. Q es un conjunto finito al cual llamaremos estados.
2. A se le llamara alfabeto de entrada.
3. Γ se le llamara alfabeto de la cinta, tenemos que $A \subseteq \Gamma$ y existira un símbolo B al cual llamaremos blanco el cual cumple $B \in \Gamma - A$. (en muchos casos $A = \{0, 1\}$ y $\Gamma = \{0, 1, B\}$).
4. δ es una función parcial llamada función de transion $\delta : Q \times \Gamma \to Q \times \Gamma \times \{\triangleright, \triangleleft\}$ donde $\triangleright$ simboliza derecha y $\triangleleft$ simboliza izquierda.
5. $q_0 \in Q$ es un estado el cual se le llamara estado inicial
6. $F \subseteq Q$ es un conjunto finito de estados llamados estados finales.

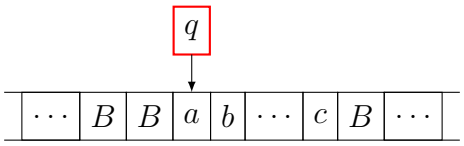
Ahora procederemos a explicar el funcionamiento de una máquina de Turing, la máquina de Turing puede entenderse como un experimento mental donde uno simula un algoritmo, imaginen a la máquina de Turing como un objeto físico el cual posee una cinta infinita en ambas direcciones dividida en múltiples celdas, en esta cinta uno puede leer la información escrita ahí como reescribirla, la escritura como lectura dependen de un cabezal de lectura el cual lee solo una celda a la vez y el mecanismo que determina cómo actuará

la máquina dependiendo de qué símbolo está leyendo depende de sus estados, de la función de transición, de su estado inicial y de sus estados finales, la idea es que con estas indicaciones tan libres sobre cómo es una máquina de Turing uno pueda realizar cualquier algoritmo que se le ocurra con ella.

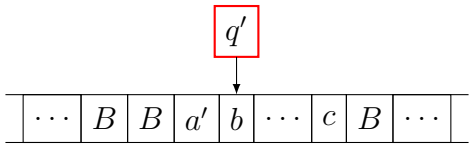
Su funcionamiento queda explicado de la siguiente manera: Supongamos que tomamos una palabra $w = w_0 \cdots w_n$ sobre un alfabeto A, hacemos que cada letra este escrita en una celda de la cinta y a la izquierda y derecha de la palabra ponemos símbolos en blanco, el cabezal de lectura se ubica al inicio de la palabra w, en este ejemplo se ubica justo encima de w_0 y comienza en el estado inicial q_0, siempre se le representa como una flecha que apunta al símbolo que esta leyendo y se puede mover a la izquierda o a la derecha de la cinta, esto se puede visualizar mediante la siguiente ilustracion:



Para comprender lo que hace la maquina de Turing y como se relacionan sus elementos, veamos lo que hace en cada paso, supongamos que el cabezal esta leyendo $a \in A$ mientras esta en un estado q.



A continuacion la maquina busca si hay alguna instruccion para (q, a), si no existe tal instruccion la maquina se detiene, en cambio si existe la maquina lo que hace es usar la función de transición δ y hace el siguiente mapeo $(q, a) \overset{\delta}{\mapsto} (q', a', D)$ con $D \in \{\triangleright, \triangleleft\}$ lo que se puede leer como "la maquina pasa del estado q al estado q', reescribe el símbolo a por el símbolo a' y el cabezal de lectura se mueve un espacio en la direccion D", si suponemos que $D = \triangleright$ (es decir, derecha) el dibujo anterior ahora se ve así.



Notemos que todo esto depende de la función de transiciónδ, la cual es una función parcial ya que no todas las configuraciones posibles de (q, a) están definidas en la función, y las que sí lo están solo van a una única configuración posible, es decir, el mapeo $(q, a) \overset{\delta}{\mapsto} (q', a', D)$ está únicamente determinado, por eso se le llama máquina de Turing determinista. Si la máquina se detiene en algún estado final $q \in F$ decimos entonces que la máquina acepta la palabra y si se detiene en un estado no final la rechaza. Entonces, asumiendo que todo funcióna de modo ideal, tomamos una palabra w, la ponemos en la máquina, cabezal en posición inicial y en estado inicial y dejamos que la máquina ejecute todas las instrucciones que tenga disponibles hasta que inevitablemente termina. Si al terminar está en un estado final, entonces la acepta (Notar que este funcionamiento no siempre es el que ocurrira, pero es un buen indicativo para entender que es lo que ocurre).

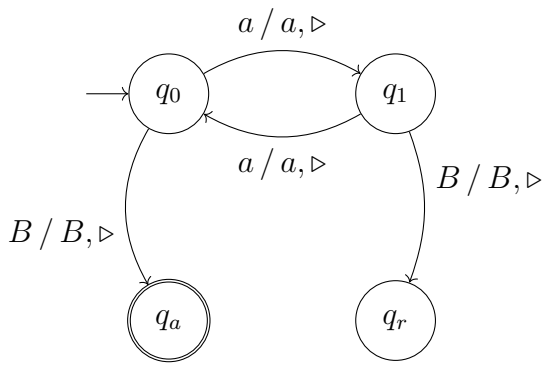
Definición 4.3. dada una maquina de turing determinista $M = (Q, A, \Gamma, q_0, F, \delta)$ definimos su lenguaje como $L(M) = \{w \in A^* : M \text{ acepta } w\}$

Observación 7. A priori, dada una palabra w una maquina de turing podria no detenerse nunca, podria quedarse perfectamente en un bucle de instrucciones del estilo $(q, a) \mapsto (q, a)$.

Ejemplo 4. *Dado $A = \{a\}$, $\Gamma = \{a, B\}$, mi conjunto de estados $Q = \{q_0, q_1, q_a, q_r\}$ y la función de transición viene dada por*

- $\delta(q_0, a) = (q_1, a, \triangleright)$
- $\delta(q_1, a) = (q_0, a, \triangleright)$
- $\delta(q_0, B) = (q_a, B, \triangleright)$
- $\delta(q_1, a) = (q_1, a, \triangleright)$

Entonces la MT $M = (Q, A, \Gamma, q_0, F, \delta)$ se puede visualizar como:



la flecha que esta a la izquierda del q_0 es para indicar que ahí inicia, las transiciones se representan como $a/b, D$ para indicar ´´se lee a, se reemplaza por b y se mueve en la direccion D" y el doble circulo que hay alrededor de q_a es una convencion para denotar cuales son los estados finales, uno puede ver que las palabras aceptadas por M son las palabras de largo par.

Observación 8. aunque se puede describir una maquina de turing en cualquier lenguaje, lo mas simple es siempre considerar los alfabetos de entrada y cinta $A = \{0, 1\}, \Gamma = \{0, 1, B\}$ con estados $Q = \{q_1, \cdots, q_m, q_f\}$ y con un unico estado final $F = \{q_f\}$, algo que sera de utilidad mas adelante pero no lo explicare en tanto detalle porque no es importante los detalles de como funcióna es que uno puede codificar la informacion de una maquina de turing como una secuencia de $0's$ y $1's$, la forma de hacerlo es primero codificar las palabras del lenguaje de la cinta, luego las transiciónes y finalmente la maquina completa, a la secuencia completa resultante se le llamara $Cod(M)$ o $C(M)$.

Algo que sera de relevancia es que las posteriores demostraciones salvo la de indecidibilidad de $Th\mathfrak{N}$ no tendrán tanto detalle en las configuraciones o transiciónes o los lenguajes, lo que uno suele hacer en ciencias de la computación es asumir que hay una codificación razónable de los códigos, de las transiciónes y trabajan con pseudocodigos, osea descripción de que se hace en cada paso asumiendo que lo necesario para realizar cada paso funcióna, hasta que llegue el momento adecuado seguiremos ese camino.

Para poder hacer lo que deseamos es necesaria la tesis de Church Turing la cual nos dice de que todo algoritmo es equivalente a una maquina de turing, es decir toda maquina de turing describe un algoritmo y para todo algoritmo existe una maquina de turing que hace exactamente lo mismo, eso nos permite conectar las nociones de decidibilidad con las maquinas de turing.

Definición 4.4. dado un lenguaje L, decimos que es recursivamente enumerable si existe una maquina de turing determinista tal que $L(M) = L$.

Es decir si $w \in L$ entonces M acepta w. En cambio si $w \notin L$ entonces M rechaza w o M entra en un loop (no se detiene).

Definición 4.5. Decimos que un lenguaje L es decidible si existe una maquina de turing determinista L tal que $L = L(M)$ y M se detiene para toda entrada (es decir es casí igual a ser recursivamente enumerable pero no pueden existir loops).

Observación 9. Es facil ver que si un lenguaje es decidible, entonces es recursivamente enumerable pero el converso no siempre es cierto, pero eso se vera mas adelante.

Haremos una convencion por notacion viendo que si $L \subseteq A^*$ es un lenguaje entonces denotamos a su complemento como $\overline{L} = A^* - L$.

Proposición 4.6. *Si un lenguaje L es decidible, entonces $\overline{L}$ tambien es decidible.*

Demostración. Como L es decidible entonces existe MT M tal que $L = L(M)$ y se detiene para toda entrada, notemos que $w \in \overline{L}$ si y solo si $w \notin L$ y esto ocurre si y solo si M rechaza w, esto nos permite definir una maquina turing M' como sigue,

Tomando como input una palabra w:

- hacemos que simule lo que haria M con la entrada w
- si M rechaza w, aceptar w
- si M acepta w, rechazar w

Como M se detiene siempre son los unicos resultados posibles, así podemos ver que $w \in L(M')$ si y solo si M rechaza w y esto es si y solo si $w \notin L$ por lo tanto $\overline{L} = L(M')$ y por ende es decidible. ■

Teorema 4.7. *Un lenguaje L es decidible $\iff L, \overline{L}$ son recursivamente enumerables.*

Demostración.

- ($\Longrightarrow$) Si L es decidible en particular es recursivamente enumerable, por proposición anterior $\overline{L}$ es decidible y por tanto recursivamente enumerable.

- ($\Longleftarrow$) Si $L, \overline{L}$ son recursivamente enumerables entonces existen maquinas M, M' tales que $L(M) = L, \overline{L} = L(M')$.Con esto podemos construir una maquina N que funcióna como sigue, dada una entrada w :

  - Simula M y a M' al mismo tiempo.
  - si M acepta w entonces N acepta w.
  - si M' acepta w entonces N rechaza w.

  Notemos que como los lenguajes son complementarios entre si, son los dos unicos casos posibles (si M no acepta w entonces debe ser aceptado por M' y viceversa), y como nunca tocamos la posibilidad de que entren en un loop esta maquina se detiene para toda entrada, y dado que acepta solo si M acepta tenemos que $L = L(M) = L(N)$ por lo que L es decidible.

Observación 10. aca usamos algo de trampa pues no explicamos que quiere decir con que una maquina simule dos al mismo tiempo ¿eso es posible? la respuesta es si, como discutimos al principio de la sección este modelo de maquina de turing es equivalente con el de multiples cintas, especificamente con las de dos por lo que podemos hacer que cada una ejecuten el codigo de cada una y así funciónen, la razón por la que hago este remark es que va a ser muy util la nocion de que una maquina simule lo que hace otra y se ve reflejado en la siguiente definicion.

Definición 4.8. Definimos una maquina de turing universal U como una maquina que sobre una entrada $z \in \{0, 1\}^*$ hace lo siguiente:

- si z tiene la forma $z = C(M)w$ con M alguna maquina de turing, entonces U simula lo que hace M con la entrada w
- Si z no tiene la forma anteriormente descrita entonces rechazo.

■

Observación 11. Definimos entonces el lenguaje de la MT universal como $L(U) = \{codMw : w \in L(M)\}$, esta noción es muy poderosa y nos sirven para definir y resolver toda serie de problemas, y es la idea detrás de todos los celulares, computadores y dispositivos electrónicos modernos, sabemos que hay un algoritmo para la suma, para la resta, para la multiplicación, para la división, etc. y seria muy impráctico tener una maquina o aplicación para cada operación así que lo que hacemos es una que pueda ejecutarlas a todas a la vez, nuestro celular tiene un comportamiento de maquina de turing universal porque

cuando nos metemos a la calculadora lo que hace es simular lo que estas maquinas harían, y así funcióna cuando corremos cualquier programa, así no necesitamos una maquina para cada algoritmo, solo necesitamos una.

Definición 4.9. Definimos le problema de la detencion de las maquinas de turing como un lenguaje llamado $Halting$ el cual se define como sigue:

$$HALTING = \{C(M)w : M \text{ es una MT }, w \in \{0,1\}^* \text{ y } M \text{ se detiene sobre } w\}$$

Siempre se supo que habían mas problemas indecidibles que decidibles por un argumento de cardinalidad pero no habían encontrado uno, fue turing en 1936 que con el uso de estas maquinas pudo demostrar que un problema era indecidible de manera muy elegante y simple, el problema anteriormente descrito $HALTING$ es recursivamente enumerable pero no decidible.

Teorema 4.10. *$HALTING$ es indecidible*

Demostración. Supongamos que es decidible, por lo tanto existe una maquina H tal que para toda maquina de turing M se tiene que:

- Si M se detiene sobre w entonces H acepta $C(M)w$
- Si M no se detiene sobre w entonces H rechaza $C(M)w$

A partir de H podemos definir una nueva maquina N que funcióne de la siguiente manera, dada una entrada $z \in \{0,1\}^*$:

- Si z no es de la forma $C(M)$ para alguna maquina de turing, rechaza
- Si $z = C(M)$ para alguna maquina de turing, entonces N simulara lo que hace H sobre la entrada $zz = C(M)C(M)$ de donde tenemos que N se comportara de la siguiente manera:
    1. si H acepta (M se detiene sobre $C(M)$) entonces N entra en un loop (no se detiene).

2. Si H rechaza (M no se detiene sobre $C(M)$) entonces N acepta.

Ahora ¿que pasara si a N le damos como entrada $C(N)$?. Si N acepta $C(N)$ significa que H rechazo $C(N)C(N)$ lo cual implica que N no se detiene sobre $C(N)$.Y si N no se detiene al recibir la entrada tenemos que H acepta $C(N)C(N)$ lo que significa que N se detiene sobre $C(N)$ por lo que concluimos que

$$N \text{ se detiene sobre } C(N) \iff N \text{ no se detiene sobre } C(N).$$

Por lo que concluimos que $HALTING$ no puede ser decidible. ■

Observación 12. La clave para la contradicción es que hacemos que N determine que es lo que ocurrirá a partir de su propio código lo que si misma hará, ella debe determinar lo que hará un sistema del que forma parte, formando una paradoja de autoreferencia.

Teorema 4.11. *$HALTING$ es recursivamente enumerable*

Demostración. Definimos una maquina de turing A de la siguiente forma, dada una entrada $z \in \{0,1\}^*$:

1. si $z \neq C(M)w$ para alguna maquina M y $w \in \{0,1\}^*$ entonces A rechaza
2. si $z = C(M)w$ para alguna maquina M y $w \in \{0,1\}^*$ entonces A simula M sobre w, si la simulacion se detiene entonces A acepta

es sencillo ver a partir de esta descripcion que $L(A) = HALTING$. ■

Observación 13. Con esto podemos ver que $decidible \subsetneq recursivamente\ enumerable$

Hay dos formas de demostrar que un problema es o no es decidible, la primera es intentarlo a mano lo que no es nada práctico y la segunda es hacer una reducción, se estudia un lenguaje L y de alguna forma tú lo conviertes en otro lenguaje K que ya conoces de tal manera que si puedes resolver K, entonces resuelves L y esa estrategia es la que se tomará para demostrar la indecidibilidad de $Th\mathfrak{N}$.

Definición 4.12. Decimos que una MT M computa una función $f : A^* \to A^*$ si:

- para todo $w \in A^*$ la MT se detiene y acepta.
- al detenerse, el contenido de la cinta es $f(w)$.

Diremos que la función es computable si existe una maquina que la computa.

Definición 4.13. Dados dos lenguajes $L, K \subseteq A^*$,una reducción desde L a K es una función $f : A^* \to A^*$ computable tal que para todo $w \in A$

$$w \in L \Longleftrightarrow f(w) \in K.$$

Observación 14.

1. la función que nos da la enumeración no tiene porque ser inyectiva, sobreyectiva o biyectiva, lo importante es que nos permita reducir el lenguaje L a K.
2. Si existe una reducción de L a K se le denota como $L \leq_m K$.
3. La relación $\leq_m$ es transitiva.

Teorema 4.14. *Si $L \leq_m K$ y K es decidible entonces L es decidible.*

Demostración. Si $L \leq_m K$ entonces existe una función computable $f : A^* \to A^*$ tal que $w \in L \Longleftrightarrow f(w) \in K$. Como K es decidible entonces existe M_K tal que $L(M_K) = K$ y M decide K, por la otra hipotesis al f ser computable existe una M_f que computa f.

Construiremos una MT M_L que decida L, dada una entrada $w \in \{0, 1\}^*$:

- hacemos que M_f compute w para convertirlo en $f(w)$.
- hacemos que se simule M_K sobre $f(w)$, si M_K acepta, entonces M_L acepta y si M_K rechaza entonces M_L rechaza,

Como M_K se detiene para toda entrada tenemos que M_L tambien, lo unico que falta demostrar es que $L = L(M_L)$, lo probaremos por doble contencion:

1. ($\subseteq$) si $w \in L$ entonces $f(w) \in K$, luego hacemos que M_L reciba a w como entrada, como $f(w) \in K = L(M_K)$ cuando M_K simule $f(w)$ tenemos que se detendra, haciendo que M_L acepte w concluyendo $w \in L(M_L)$.

2. ($\supseteq$) Si $w \in L(M_L)$ significa que M_K se detiene sobre $f(w)$, esto implica que $f(w) \in L(M_K) = K$ lo cual implica que $w \in L$.

■

Corolario 4.15. *Si $L \leq_m K$ y L es indecidible, entonces K es indecidible.*

Demostración. resultado directo usando la contrapositiva del teorema anterior. ■

Antes de proseguir con una demostracion conaidero necesario ver un ejemplo en que esta reducción se ve para determinar la decidibilidad o indecidibilidad de problemas.

Proposición 4.16. *Definimos el problema $NON - EMPTY$ como*

$$NON - EMPTY = \{C(M) : M \textit{ es una MT y } L(M) \neq \varnothing\}$$

tenemos que $NON - EMPTY$ es indecidible (es decir, el determinar si el lenguaje de una maquina de turing es o no vacio es indecidible).

Demostración. Haremos una reducción desde $HALTING$ definiendo una función $f : A^* \to A^*$ tal que

1. Si $z \neq C(M)$ entonces $f(z) = 0 \notin NON - EMPTY$.
2. si $z = C(M)w$ definimos $f(z) = f(C(M)w) = C(N_{M,w})$ donde $N_{M,w}$ es una maquina de turing tal que para toda entrada u :
   - Borra la entrada u
   - Escribe w en la cinta
   - Simula M sobre w, si M se detiene sobre w acepta.

Probaremos que $w \in HALTING \iff f(w) \in NON - EMPTY$. Si $w \in HALTING$ entonces $w = C(M)v$ entonces $f(w) = C(N_{M,v})$ para demostrar que esta en $NON - EMPTY$ es suficiente mostrar que $L(N_{M,v}) \neq \varnothing$.

Sea $u \in \{0,1\}^*$, al ponerla como entrada en $N_{M,v}$ tenemos que simula lo que hace M sobre v, como $w \in HALTING$ tenemos que M se detiene sobre w por lo tanto acepta y así $L(N_{M,v}) \neq \varnothing$.

La otra direccion es analoga, si $f(w) = C(N_{M,v}) \in NON{-}EMPTY$ entonces $L(N_{M,v}) \neq \varnothing$ lo cual es cierto si M se detiene sobre v, i.e $w \in HALTING$.

Tenemos que $HALTING \leq_m NON - EMPTY$, como $HALTING$ es indecidible tenemos que $NON - EMPTY$ es indecidible. ■

5. Demostración de la indecidbilidad de $Th\mathfrak{N}$

En esta sección nos dedicaremos al completo en una demostracion de la indecidibilidad de la teoría aritmética usando maquinas de turing, con lo ya visto uno puede intuir como sera la demostracion, haremos una reducción desde $HALTING$, para lograr esta hazaña se hara uso de las funciónes parciales recursivas y la función β de Gödel.

Definición 5.1. Definimos a las funciónes parcialmente recursivas como toda función m-aria de $\mathbb{N}^m$ a $\mathbb{N}$ con $m \geq 0$ que puede ser construida de modo inductivo a partir de las siguientes funciónes y reglas:

1. $f(x) = S(x)$ donde S denota al sucesor de x.
2. $f(x_1, \cdots, x_n) = k$ con $k \in \mathbb{N}$ fijo (las constantes son funciónes parcialmente recursivas).
3. $f(x_1, \cdots, x_n) = x_i$ tal que $1 \leq i \leq n$ (las proyecciones).
4. Si $f(x_1, \cdots, x_n) = g(h_1(x_1, \cdots, x_n), \cdots, h_m(x_1, \cdots, x_n))$ donde $g, h_1, \cdots, h_m$ son funciónes parcialmente recursivas (están preservadas bajo composición).
5. $f(0) = k, f(S(x)) = g(x, f(x))$ donde $k \in \mathbb{N}$ y g es una función parcialmente recursiva (como veremos mas adelante la suma y producto en $\mathbb{N}$ son funciónes parcialmente recursivas al definirse con una estructura de ese estilo).
6. $f(0, x_2, \cdots, x_n) = g(x_2, \cdots, x_n), f(S(x), x_2, \cdots, x_n) = h(x, f(x, x_2, \cdots, x_n), x_2, \cdots, x_n)$ donde g, h son funciónes parciales recursivas (una version generalizada del punto anterior).

Ejemplo 5. *Sabemos que podemos definir la suma y el producto como funciónes parcialmente recursivas gracias a que se definen en función del sucesor que es recursiva y ademas:*

1. *Definimos la suma en* $\mathbb{N}$ *como*

$$a + 0 = a, a + S(b) = S(a + b)$$

   la cual adquiere una estructura como en el punto 5 obteniendo que es recursiva.

2. *Definimos el producto en* $\mathbb{N}$ *como*

$$a \cdot 0 = 0, a \cdot S(b) = a \cdot b + a$$

   la cual adquiere una estructura como en el punto 5, luego como es composición de suma y sucesor que son recursivas tenemos que el producto tambien es recursivo.

La razón de porque queremos hacer esto es porque, sea $\mathcal{L} = \{0, 1, s, +, \cdot, <\}$ queremos demostrar que cualquier función recursiva se puede definir en $\mathfrak{N}$, es decir, para toda $f : \mathbb{N}^k \to \mathbb{N}$ recursiva existe una $\mathcal{L}$-oracion $\varphi_f(x_1, \cdots, x_k, y)$ tal que para todo $a_1, \cdots, a_k, b \in \mathbb{N}$ se tiene que

$$f(a_1, \cdots, a_k) = b \Longleftrightarrow \mathfrak{N} \models \varphi_f(a_1, \cdots, a_k, b)$$

Es decir, la función nos da un valor concreto si y solo si su formula lógica que lo representa es verdadera en la aritmética, esto nos permitiria hacer el salto de la aritmética a la lógica y poder abordar el problema. La función β de Gödel nos servira para codificar cualquier función recursiva en $\mathbb{N}$.

Definición 5.2. Dados $a, b, c \in \mathbb{N}$ definimos la función beta de Gödel $\beta : \mathbb{N}^3 \to \mathbb{N}$ como sigue:

$$\beta(a, b, c) = a \quad \text{mód } [b(c + 1) + 1].$$

Ejemplo 6. $\beta(32, 7, 2) = 32 \ \text{ mód } 7(2 + 1) + 1 = 32 \ \text{ mód } 22 = 10$

la gracia de esta función y porque nos interesa es que nos permite iterar y nos permite para todo natural expresarlo como la imagen de una tripleta de valores.

Lema 5.3. *dada una sucesión $r_0, r_1, \cdots, r_n$ de números naturales existen unos números $a, b \in \mathbb{N}$ fijos tales que:*

$$\beta(a, b, i) = r_i \; \forall i \; 1 \leq i \leq n$$

Es decir, para toda sucesión finita de naturales existen un par de naturales fijos que permiten describir a toda sucesión en terminos de la función β de Gödel.

Ejemplo 7. *Dada la secuencia $r_0 = 2, r_1 = 6, r_2 = 3$ tenemos que:*

$$\beta(1{,}651{,}320{,}463{,}326\ , 720, 0) = 2$$
$$\beta(1{,}651{,}320{,}463{,}326\ , 720, 1) = 6$$
$$\beta(1{,}651{,}320{,}463{,}326\ , 720, 2) = 3$$

Demostración. Considere la sucesión de naturales $r_1, \cdots, r_n \in \mathbb{N}$, definimos $b = (max\{n, r_1, \cdots, r_n\})!$, notemos que:

- $b \geq n, b \geq r_i$ para todo $1 \leq i \leq n$.
- $k|b$ para todo $0 < k \leq n$. La razón de esto es que si $b = r!$ con $r = max\{n, r_1, \cdots, r_n\}$ tenemos que si $r = n$ es directo, si no $r - n > 0$, entonces $b = r(r-1)\cdots(n+1)n!$y ahí tenemos el mismo resultado.

Definimos para cada $i \in \mathbb{N}$ $m_i = b(i+1) + 1$ vamos a demostrar que para todo i, j tales que $0 \leq i < j \leq n$ son coprimos. Por contradicción supongamos que no lo son, entonces existe un primo p tal que $p|m_i$ y $p|m_j$.

Entonces $p|(m_j - m_i) = b(j-i)$ por lo tanto $p|b$ o $p|(j-i)$. Si $p|b$ como $p|m_i$ en particular $p|m_i - b(i+1) = 1$ lo cual es una contradicción por lo que necesariamente $p|(j-i)$.

Dado que $0 < j - i < j \leq n$ tenemos que $j - i|b$ por lo que si $p|(j-i)$ por transitividad tenemos que $p|b$ lo que tambien es una contradicción, por lo tanto tenemos que m_i, m_j son coprimos para todo $i \neq j$.

Como son coprimos, usando el teorema chino del resto tenemos que como $mcd(m_i, m_j) = 1$ para cada $0 \leq i < j \leq n$ existe un numero $a \in \mathbb{N}$ tal que $a_i \equiv r_i \mod m_i$.

Dado que para cada i se tiene que $r_i \leq b < m_i$ concluimos que $a \mod m_i = r_i$, por la construccion hecha tenemos que $\beta(a, b, i) = r_i$ para todo $i \in \{0, \cdots, r_i\}$. ■

Observación 15. Notemos que la función β se puede definir en lógica de primer orden siguiendo la siguiente formula, si:

$$\varphi_{mod}(x, y, z) = \exists u(x = u \cdot y + z \wedge 0 \leq z \wedge z < y)$$

Notemos que $\varphi_{mod}(x, y, z)$ es cierta si y solo si $x \equiv z \mod y$. Entonces podemos definir la función β de Gödel en lógica de primer orden a traves de la siguiente formula:

$$\varphi_\beta(x, y, z, w) = \varphi_{mod}(x, y \cdot (z + 1) + 1, w)$$

En efecto $\varphi_{mod}(x, y \cdot (z + 1) + 1, w)$ ocurre si y solo si $x \equiv w \mod [y \cdot (z + 1) + 1]$ lo cual ocurre si y solo si $x = \beta(w, y, z)$ obteniendo que se puede definir, esto nos permitira definir la función exponencial en lógica de primer orden y en general cualquier función recursiva.

La función exponencial es recursiva ya que se puede definir $2^0 = 1, 2^{x+1} = 2 \cdot 2^x$. y se puede definir en lógica de primer orden a traves de una función $\varphi_{exp}(y, z)$ definida a partir de φ_β. La formula $\varphi_{exp}(y, z)$en cuestion es:

$$\exists a \exists b[\varphi_\beta(a, b, 0, 1) \wedge \forall v \forall w(0 \leq v \wedge v < y \wedge \varphi_\beta(a, b, v, w) \rightarrow \varphi_\beta(a, b, S(v), 2w)) \wedge \varphi_\beta(a, b, y, z)]$$

Evaluemos porque esta formula es cierta, primero partamos del hecho de que queremos que $\varphi_{exp}(y, z)$ sea cierto si y solo si $z = 2^y$. Notemos que esto se puede hacer de manera relativamente sencilla ocupando esta formula (que se aprovecha de la manera recursiva de definir la exponencial) :

$$\exists u_0 \cdots \exists u_y[u_0 = 1 \wedge \forall v((0 \leq v \wedge v < y) \rightarrow u_{v+1} = 2u_v) \wedge z = u_y].$$

Esta formula quiere decir que $z = \underbrace{2 \cdots 2}_{y-veces.}$, sin embargo esto no es una formula en lógica de primer orden porque el numero de variables dependen del y, notemos que la formula

creo la secuencia $1, 2, 4, \cdots, 2^y = z$ por lo que por el lema existen dos a, b que permiten expresar toda la secuencia en terminos de la función β ahí el problema de que el numero de variables cambie no existe, entonces uno puede leer la formula complicada como "existen dos números a, b que describen toda la secuencia a partir de la función β de Gödel, dicha sucesión parte con 1 y termina en z y si cada numero de la sucesión la cumple, su sucesor tambien y nos permite expresar $z = 2^y$".

De manera analoga a lo visto ahí uno puede expresar cualquier función recursiva en función de β, tomamos una función recursiva $f : \mathbb{N}^k \to \mathbb{N}$, como es recursiva sabemos que tiene la forma:

$$f(x_1, \cdots, x_k, 0) = g(x_1, \cdots, x_k)$$
$$f(x_1, \cdots, x_k, s(y)) = h(x_1, \cdots, x_k, y, f(x_1, \cdots, x_k, y))$$

Con h, g recursivas, podemos suponer que están definidas por proposiciones φ_h, φ_g que las representan (podemos hacerlo ya que se construyen inductivamente), la proposición que representa a f la cual llamaremos $\varphi_f(x_1, \cdots, x_k, y, z)$ se define de la siguiente manera:

$$\exists a \exists b [\forall u (\varphi_g(x_1, \cdots, x_k, u) \to \varphi_\beta(a, b, 0, u)) \wedge$$
$$\forall v \forall w_1 \forall w_2 ((0 \leq v \wedge v < y \wedge \varphi_\beta(a, b, v, w_1) \wedge \varphi_h(x_1 \cdots, x_k, v, w_1, w_2)) \to \varphi_\beta(a, b, S(v), w_2))$$
$$\wedge \varphi_\beta(a, b, y, z)]$$

Observación 16. Es claro que todo lo que hace una función parcial puede ser hecho con maquinas de turing ya que cada operación básica que las define inductivamente puede ser hecha con un algoritmo y por ende una maquina de turing, es mas, ambos modelos son equivalentes, para una función recursiva hay una MT que hace lo mismo y viceversa.

Con esto tenemos finalmente todos los ingredientes para una demostracion de la indecidibilidad de la aritmética.

Teorema 5.4. *$Th\mathfrak{N}$ es indecidible.*

Demostración. Haremos una reducción desde $HALTING$, dada una MT M y una palabra w queremos construir una proposición $\varphi_{M,w}$ tal que

$$M \text{ se detiene en } w \iff \mathfrak{N} \models \varphi_{M,w} (\text{ i.e } \varphi \in Th\mathfrak{N}).$$

Codificamos una ejecución de la maquina de turing M, es decir una sucesiónde configuraciones que empiezan desde que w es introducida hasta que termine (una configuración es el estado actual y el contenido de la cinta) como un numero natural $n = \alpha_0 \cdots \alpha_k$ tales que:

1. Cada α_i representa una configuración de M con un numero finito de celdas seleccionadas
2. Para cada α_i, α_j se tiene que estos números tienen la misma cantidad de dígitos (son configuraciones con el mismo numero de celdas).
3. α_0 representa al configuración inicial de M (con w como entrada).
4. α_{i+1} es la configuración resultante de aplicarle a la configuración α_i la función de transición δ.
5. α_k tiene la configuración que tiene cuando se detiene (sabemos que se detiene porque tomamos una tupla (M, w) en $HALTING$).

Ahora procederemos con una codificación de las celdas, supongamos que $*$ es un símbolo especial que no aparecen en los estados, escoja un orden lineal arbitrario sobre $\{0, 1, B\} \times (Q \cup \{*\})$, a cada par $(a, q) \in \{0, 1, B\} \times (Q \cup \{*\}$ le asígnamos un numero arbitrario de la forma $1l$ con l es la posición que ocupa (a, q) en ese orden partiendo desde 0, luego agregue cuantos ceros sean necesarios para que todos esos números tengan la misma cantidad de digitos.

Ejemplo 8. *Suponga que $Q = \{q_0, q_1, q_2\}$ y suponga el siguiente orden lineal sobre los pares en $\{0, 1, B\} \times \{q_0, q_1, q_2, *\}$:*

$$(0, *) < (1, *) < (B, *) < (0, q_0) < (1, q_0) < (B, q_0) <$$
$$(0, q_1) < (1, q_1) < (B, q_1) < (0, q_2) < (1, q_2) < (B, q_2)$$

Entonces vemos que a (B, q_2) se le asígna el 111,al $(1, q_2)$ el 110 hasta llegar al $(0, *)$ que se le asígna el 100, notar que el largo de las codificación es de la celda dependen solo de M,en general el largo va a ser una cierta constante c.

Ahora procederemos a codificar las configuraciones, representaremos a cada configuración como un par $(a, q) \in \{0, 1, B\} \times (Q \cup \{*\})$ donde:

- a es el contenido de la cinta.
- Si $q = *$ entonces la cabeza lectora no se encuentra en esa celda.
- Si $q \in Q$ entonces la cabeza lectora se encuentra en la celda y M esta en el estado q.

Una configuración es representada entonces como la secuencia de números asígnados a los pares que representen sus celdas.

Ejemplo 9. *Suponga que se quiere representar una configuración de M la cual se tiene el siguiente contenido de la cinta(Omitiendo los otros valores B):*

$$B011B$$

Ademas suponga que M esta en el estado q_2 y que el cabezal esta en el 1 del medio, dicha cofnfiguracion se ve representada por la siguiente secuencia de pares:

$$(B, *)(0, *)(1, q_2)(1, *)(B, *)$$

Luego si representamos cada par con un numero como en el ejemplo anterior tenemos que se puede representar como el numero:

$$102100110101102$$

Todo este trabajo es para poder codificar todos los procesos de la maquina de turing en un numero que pueda ser entendido en $Th\mathfrak{N}$, recordemos que dada una maquina M y una palabra w queremos construir una proposición $\varphi_{M,w}$ tal que

$$M \text{ se detiene en w} \iff \mathfrak{N} \models \varphi_{M,w}.$$

Sea c el largo de las codificación es de las celdas (dependen solo de la maquina M), la idea de la estructura que la formula es que sea de la forma:

$$\varphi_{M,w} = \exists x \exists y \exists z (m_1 \wedge m_2 \wedge m_3 \wedge m_4).$$

Donde:

- m_1 signifique x es una secuencia de z bloques de largo $y \cdot c$.
- m_2 signifique el primer bloque corresponde a la configuración inicial de M sobre w.
- m_3 bloques consecutivos corresponden a configuraciones que siguen de δ.
- m_4 el ultimo bloque corresponde a la configuración de detencion.

Necesitamos formulas que nos permita extraer un bloque desde un numero en una posición dada como parametro, la razón de esto es que si podemos definir dichas formulas entonces es facil ver que podemos definir m_1, m_2, m_3, m_4 (si un numero n representa una secuencia de configuraciones, un bloque de n es una configuración y si un numero m representa una configuración, un bloque de m es una celda).

Vamos a definir dos formulas:

1. $\varphi_{bloq}(x, y, z)$ como que es cierta si x se puede dividir en y bloques de largo z.

   Para definirla considere las formulas auxiliares:

   $$\varphi_{div}(x, y, z) = \exists w (0 \leq w \wedge w < y \wedge x = z \cdot y + w).$$

   $$\varphi_{menos}(x, y, z) = \ x = y + z$$

   Ademas supongamos que la formula $\varphi_{exp10}(x, y)$ es verdad si $y = 10^x$ (se define de manera similar a φ_{exp}).

   Usando esas formulas definimos la siguiente formula:

   $$\varphi_{largo}(x, y) = \exists z \exists u \exists v (\varphi_{menos}(y, 1, z) \wedge \varphi_{exp10}(z, u) \wedge \varphi_{exp10}(y, v) \wedge u \leq x \wedge x < v)$$

   lo que finalmente nos permite definir la formula deseada como $\varphi_{bloq}(x, y, z) = \exists u (\varphi_{largo}(x, u) \wedge u = y \cdot z)$.

2. $\varphi_{bloq-ind}(x, y, z, w)$ como que es cierta si x se puede dividir en bloques de largo y, y el $z-$esimo de esos bloques es w (contando desde 0 de izquierda a derecha).

   Para definir esta formula usamos la función β de Gödel y la formula φ_{mod} definida anteriormente.

$$\exists u \exists l \exists i \exists a_1 \exists b_1 \exists a_1 \exists b_2 [\varphi_{bloq}(x, u, y) \wedge 0 \leq z \wedge z < u \wedge \varphi_{exp10}(y, l) \wedge$$
$$\forall u_1 (\varphi_{mod}(x, l, u_1) \rightarrow \varphi_\beta(a_1, b_1, 0, u_1)) \wedge$$
$$\forall u_1 (\varphi_{div}(x, l, u_2) \rightarrow \varphi_\beta(a_2, b_2, 0, u_2)) \wedge$$
$$\forall u_3 \forall u_4 \forall u_5 \forall u_6 ((0 \leq u_3 \wedge s(u_3) < u \wedge \varphi_\beta(a_2, b_2, u_3, u_4) \wedge$$
$$\varphi_{mod}(u_4, l, u_5) \wedge \varphi_{div}(u_4, l, u_6)) \rightarrow$$
$$(\varphi_\beta(a_1, b_1, s(u_3), u_5) \wedge \varphi_\beta(a_2, b_2, s(u_3), u_6))) \wedge$$
$$\varphi_{menos}(u, s(z), i) \wedge \varphi_\beta(a_1, b_1, i, w)].$$

Veamos que M se detiene en w si y solo si $\mathfrak{N} \models \varphi_{M,w}$.

- ($\Longrightarrow$) si M se detiene en w existe una sucesión de configuraciones que termina en algún momento, luego por la construccion de la coficicacion existen una secuencia de bloques de cierto largo, el primero corresponde a la configuración inicial, los subsiguientes a otras de la configuración y el ultimo a la de detencion, como todas se satisfacen tenemos que $\mathfrak{N} \models \varphi_{M,w}$
- ($\Longleftarrow$) Como $\mathfrak{N} \models \varphi_{M,w}$ significa que se satisfacen m_1, m_2, m_3, m_4. lo que significa que existen bloques que representan la computación de M sobre w y que este termina en algún momento por lo que M se detiene sobre w.

Así concluimos que $HALTING \leq_m Th\mathfrak{N}$ por lo que $Th\mathfrak{N}$ es indecidible. ■